Assignment 3 — Breaking the Ratchet

Technical Report

Roll No. 23b1023 | Roll No. 23b0945

CS6008 — Network Security

Abstract

Investigation of a deliberately flawed Double Ratchet + X3DH end-to-end messaging implementation (Docker image `s4ch1n/as3:latest`), with three C/C++ proof-of-concept attacks built and verified in a controlled lab. All three attacks are complete. **Attack 0** shows that an unauthenticated header integer drives unbounded pre-authentication key derivation and storage, giving a single-frame denial-of-service. **Attack 1** shows that `fetch_bundle` consumes a one-time prekey without authentication or rate limiting, letting an ordinary client deny all new sessions to an offline user. **Attack 2** demonstrates a malicious server performing a full, transparent, bidirectional man-in-the-middle via identity-key substitution, reading the entire conversation while neither party detects anything. Each section gives the analysis, demonstrated behaviour, measured evidence, and a fix in Python-style pseudocode.

1 Environment

Three containers on a shared Docker network (`as3net`) from image `s4ch1n/as3:latest`: `as3_server` (in-memory relay, TCP :9000, pull-based `send`/`poll`) and two clients `alice`, `bob`. Wire format is newline-delimited JSON, so headers are readable and editable on the wire. Server state is in-memory (`docker compose down` resets it); client keys **and live ratchet sessions** persist in the `as3state` volume. See `protocol.md` for the full wire format and reference pseudocode.

Clients can be pointed at an arbitrary host/port with `-server-addr` / `-port`, which is what lets us insert a man-in-the-middle between a client and the server.

2 State Diagram

The hand-drawn diagram is submitted separately as the file `state_diagram` (not reproduced here). This section records, for reference and for the viva, the symbolic transitions it shows. Assume Bob has published a bundle and Alice initiates. Required sequence:

1. `A1` Alice→Bob
2. `A2` Alice→Bob (same DH ratchet key as `A1`)
3. `B1` Bob→Alice (Bob's new DH ratchet key)
4. `B2` Bob→Alice (same DH key as `B1`)
5. `A3` Alice→Bob (Alice's new DH ratchet key)

6. B3 Bob→Alice (Bob's next DH ratchet key)

A DH-ratchet step happens exactly when the **sending side flips**. Each sender's sending chain is the peer's receiving chain.

Msg	Sender	header.dh	Transition type	Root key	Chain key	Message key
A1	Alice	A0	session-start DH ratchet, then symmetric	SK→RK0	CK_A0 (Alice send)	MK_A1 = CK_A0[0]
A2	Alice	A0 (same)	symmetric-chain advance	RK0	CK_A0	MK_A2 = CK_A0[1]
B1	Bob	B0	DH ratchet (Bob's send flips)	RK0→RK1	CK_B0 (Bob send)	MK_B1 = CK_B0[0]
B2	Bob	B0 (same)	symmetric-chain advance	RK1	CK_B0	MK_B2 = CK_B0[1]
A3	Alice	A1 (new)	DH ratchet (Alice's send flips)	RK1→RK2	CK_A1	MK_A3 = CK_A1[0]
B3	Bob	B1 (new)	DH ratchet (Bob's send flips)	RK2→RK3	CK_B1	MK_B3 = CK_B1[0]

Chain correspondence. CK_A0 is Alice's sending chain and Bob's receiving chain for A1/A2; CK_B0 is Bob's sending chain and Alice's receiving chain for B1/B2; and so on. The lab's per-message logs (dh=...tail, n, pn) confirm this structure — a repeated dh tail means a symmetric step, a new dh tail means a DH ratchet.

Granularity note. The table above counts one root-key epoch per DH ratchet. The hand-drawn diagram counts each individual KDF_RK call (two per DH ratchet: one for the receiving chain, one for the sending chain), so its root-key indices advance twice as fast (RK0 . . . RK5); both views describe the same transitions.

3 Attack 0 — Message-counter manipulation

Scenario. Mallory is an active man-in-the-middle on the wire between as3_server and Bob (Bob connects to Mallory's listen port; Mallory dials the real server). When a frame addressed to Bob passes through, Mallory inflates the header message number so the receiver is pushed far ahead of its current chain position. Implementation: attack0/attack0.cpp.

3.1 Analyze message processing

The message number n and previous-chain length pn are plaintext integer fields inside each frame header (header.n, header.pn). The server relays frames opaquely, so an on-path attacker can rewrite them. On receipt the client uses them to drive skip_message_keys, which derives and stores one message key per skipped index to support out-of-order delivery:

```
def skip_message_keys(st, until):
    if until - st.Nr > MAX_SKIP:              # intended guard (MAX_SKIP = 1000)
        raise Error("too many skipped messages")
    while st.Nr < until:
        mk, st.CKr = kdf_ck(st.CKr)
        st.skipped[(st.DHr, st.Nr)] = mk      # stored for later arrival
```

```
        st.Nr += 1
```

So a single attacker-controlled integer dictates how many key derivations and map insertions the receiver performs.

3.2 Does tampering break authentication, or influence processing? (Both)

Tampering the counter is always detected. Every tampered frame we sent — at gaps of 500, 999, 1001, 5000 and in the flood runs — produced `AEAD authentication failed`. No forged message was ever accepted, so **confidentiality and integrity hold**.

Mechanism (corrected by the Attack 2 measurement). The spec pseudocode writes `ad = st.AD + serialize(header)`, which suggests the header is bound into the AEAD associated data. Our Attack 2 format-discovery (see §5) empirically established that the real implementation's associated data is **only** the two identity DH keys — $\texttt{AD} = \texttt{ik_dh}_{\texttt{initiator}} \parallel \texttt{ik_dh}_{\texttt{responder}}$ (64 raw bytes) — and does **not** include the serialized header. So counter tampering is detected not through an AD mismatch but through the **key schedule**: `n` controls how many `kdf_ck` steps the receiver takes, so a tampered `n` derives a *different message key* and the Poly1305 tag fails under the wrong key. The observable result (`AEAD authentication failed` on every tamper) and all conclusions below are unchanged; only the reason the tag fails is restated accurately.

But the attacker still influences processing, because of three cooperating implementation flaws that permit a resource-exhaustion (availability) attack:

1. **Verify-after-work ordering (core flaw).** In `ratchet_decrypt`, the skip loop runs *before* the AEAD tag is checked, so the attacker-chosen amount of CPU/memory is spent up front and only *then* is the frame rejected.

2. **The skip bound is not enforced.** The spec caps skips at `MAX_SKIP = 1000`, but the lab image does not. We forced single-step gaps of 999, 1001 and 5000 — all did the full work and failed AEAD, and "too many skipped messages" **never fired**. The gap is effectively unbounded. (This is a concrete divergence of the implementation from the specification.)

3. **No rollback on failure.** `Nr`, `CKr`, and the stored skipped keys persist even though authentication failed — we observed `Nr` marching forward across consecutive *failed* frames, and memory that plateaus and never drops.

Summary for grading. Modifying the counter *does* invalidate the AEAD tag, so no forged message is accepted; the resource-exhaustion is permitted by a separate flaw — the receiver performs unbounded skipped-key derivation and storage *before* verifying the tag, and never rolls that state back on failure. An unauthenticated header integer thus controls an arbitrary amount of pre-authentication CPU and memory work.

Assumptions. Mallory is an active MITM on the Bob↔server link; the transport framing is plaintext JSON (header readable/editable); only the payload and AD are authenticated, not the transport's exposure of the header to tampering.

Boundary experiments (evidence for B)

Injected gap (fresh chain, `Nr=0`)	Observed on Bob	Interpretation
`-gap 999`	`AEAD authentication failed`	∼999 keys derived, then auth fails
`-gap 1001`	`AEAD authentication failed`	work done — **cap is not 1000**
`-gap 5000`	`AEAD authentication failed`	∼5000 keys derived — no cap hit

The absolute `n` is not what the guard bounds — the cost is the gap `n − Nr` at receipt. Within one chain a fixed offset has diminishing returns (once `Nr` jumps forward, the next same-chain frame skips nothing); each peer DH-ratchet resets `Nr` to 0 and re-arms a full-size skip. This motivated the amplifier modes below.

3.3 Measure the impact

To make the cost visible without relying on the peer sending many messages, `attack0.cpp` adds a **flood** mode: from a single genuine Alice message Mallory emits `K` separate frames with stepped `n` (`N`, `2N`, . . . , `K·N`), so Bob derives and stores ≈ `K·N` message keys in one burst. Bob's CPU and memory were sampled with `measure/capture.sh` (via `docker stats`) and plotted with `measure/plot.py`.

Setting	Keys forced	Peak CPU	Resident memory	Notes
idle baseline	—	∼0.15%	∼2.6–10 MiB	—
`-flood 100 -gap 10000`	1,000,000	∼9%	≥114 MiB (capture truncated mid-ramp)	memory held
`-flood 100 -gap 100000`	10,000,000	**51–177%** (multi-core)	**2.60 GiB**, held after the burst	∼40 s of CPU saturation from one message

Findings:

- **Linear, attacker-controlled cost: ≈270 bytes per stored key** (from the plateaued 10M run) and ≈250k derivations/s. 10× the keys → ∼10× the memory.
- **Permanent damage (no rollback):** in the 10M run memory climbs to 2.60 GiB, then CPU returns to idle while memory **stays flat at 2.60 GiB** — the skipped keys are never freed.
- **Denial of service:** a *single* unauthenticated frame bought ∼40 s of Bob's CPU (fully saturating ≥1 core) plus gigabytes of permanently-held memory, during which Bob cannot service legitimate traffic.
- **OOM extrapolation:** at ∼270 B/key the 6.929 GiB container limit is reached at ∼25M keys (`-flood 100 -gap 250000`), i.e. Bob is OOM-killed. The linear scaling across the 1M and 10M points justifies the extrapolation.

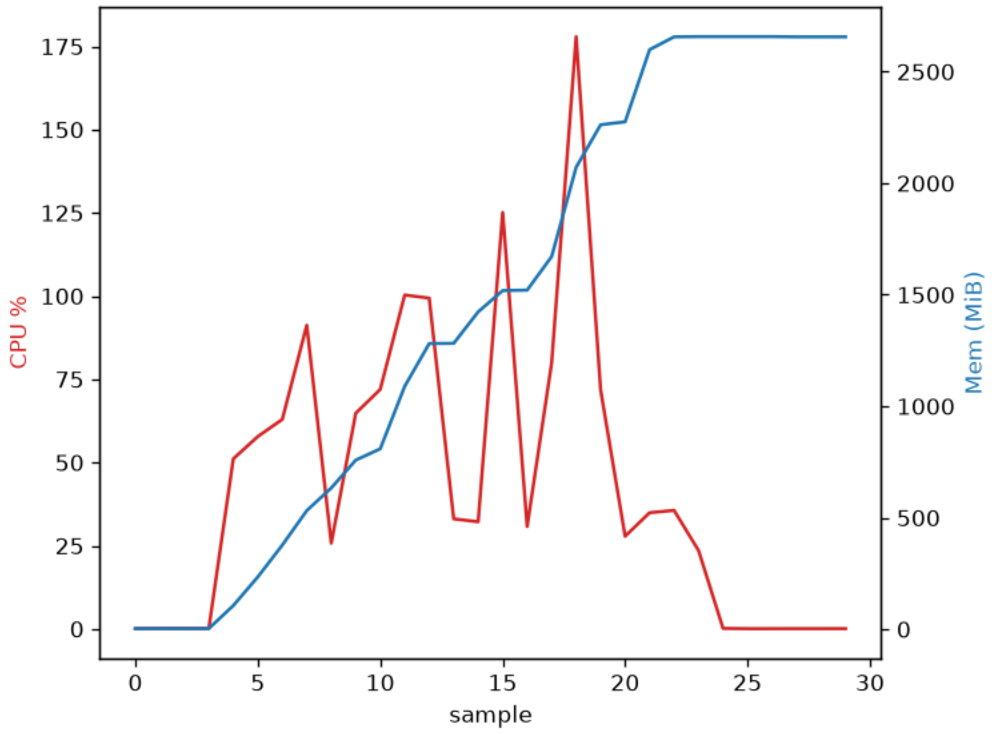


Figure 1: Headline run (10M forced keys): resident memory ramps to 2.6 GiB and **plateaus** while CPU drops back to zero — the no-rollback signature. One unauthenticated frame causes permanent, gigabyte-scale memory retention.

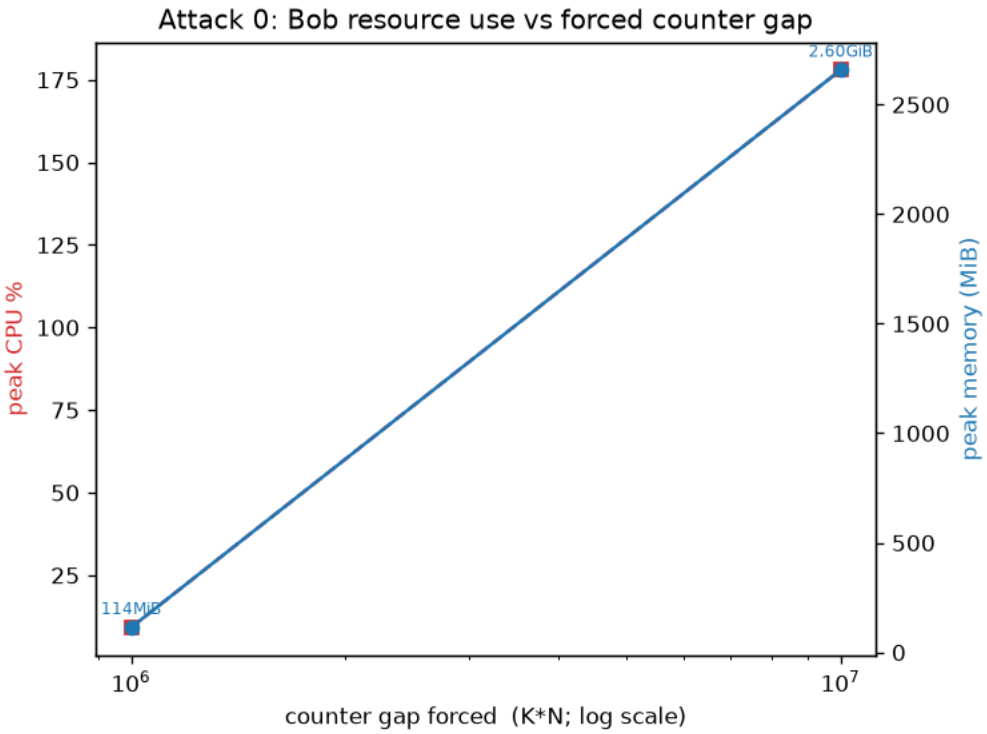


Figure 2: Peak memory and peak CPU versus the forced gap (`K·N`, log x-axis), showing the linear attacker-controlled cost. A companion 1M-key time series is in `flood_1000000.png`.

3.4 Propose a fix (pseudocode)

```
def ratchet_decrypt(st, header, ct):
    mk = try_skipped_keys(st, header)
    if mk is not None:
        return aead_decrypt(mk, ct, st.AD + serialize(header))   # may raise

    # (1) HARD-BOUND the skip BEFORE doing any work (reject, don't derive).
    gap    = header["n"]  - st.Nr
    pn_gap = header["pn"] - st.Nr if header["dh"] != st.DHr else 0
    if gap > MAX_SKIP or pn_gap > MAX_SKIP:
        raise Error("too many skipped messages")   # cheap, no derivation

    # (2) Work on a COPY; commit only if authentication SUCCEEDS (rollback-safe).
    trial = st.clone()
    if header["dh"] != trial.DHr:
        skip_message_keys(trial, header["pn"])
        dh_ratchet(trial, header)
    skip_message_keys(trial, header["n"])
    mk, trial.CKr = kdf_ck(trial.CKr); trial.Nr += 1

    pt = aead_decrypt(mk, ct, trial.AD + serialize(header))  # raises on forgery
    st.commit(trial)                 # only reached if auth PASSED
    return pt

# (3) Also cap the TOTAL stored skipped keys across the session (bounded cache,
#     evict oldest), so even a sequence of valid-looking gaps cannot grow memory
#     without end.
```

The three fixes map to the three flaws: **(1)** enforce the skip bound and check it *before* deriving; **(2)** verify the AEAD tag before committing any ratchet state, so a forged header does no lasting work and nothing is stored on failure; **(3)** bound the skipped-key store globally.

4 Attack 1 — Prekey Pogo (OPK exhaustion)

Scenario. Mallory is an *ordinary client* with no special network position — only the ability to open a TCP connection to `as3_server`. Bob has published his prekey bundle and gone offline. Mallory repeatedly calls `fetch_bundle` for Bob; each call consumes one of Bob's one-time prekeys (OPKs). Once the pool is empty, no new party can start a session with the offline Bob until he republishes. Implementation: `attack1/attack1.cpp`, built and verified in stages (single fetch → single-connection drain loop → concurrency/consequence probes).

4.1 OPK lifecycle (generation, storage, consumption, replenishment)

- **Generation / publish.** Bob generates a fixed batch at startup and `publish` uploads them: the server logs `publish: bob (+5 OPKs, total 5)`. The batch size is the lab's `OPK_COUNT = 5`.
- **Storage.** The server keeps a per-user pool of OPKs, each with an integer `opk_id`. The bundle it returns names the specific `opk_id` handed out.
- **Consumption — at fetch, not at first message.** Each `fetch_bundle` *pops* one OPK: the reported `count` walks $4 \to 3 \to 2 \to 1 \to 0$ across five fetches, and the server logs

`fetch_bundle: bob (OPKs remaining: N)` for each — all *before Mallory ever sends a message*. So consumption is bound to the fetch request, giving an attacker a direct, message-free drain primitive.

- **Allocation order.** `opk_id`s are handed out sequentially `1,2,3,4,5` and never repeat within a run (simple pop from the pool).
- **Replenishment — none automatic.** The server does not regenerate OPKs; the pool only refills when Bob comes back online and re-publishes. While Bob is offline the pool is monotonically non-increasing.

4.2 The weaknesses that permit the attack

Three independent server-side weaknesses, each confirmed empirically:

1. **`fetch_bundle` is unauthenticated.** `./attack1 -no-register` drains all five OPKs with **`no register:` line** in the server log — an anonymous socket that never identifies itself still consumes Bob's prekeys. The server never checks *who* is asking or whether the requester is a plausible session peer.
2. **No rate limiting.** A single connection pulls the entire pool back-to-back; the five successful fetches and the subsequent denial all land in the **same one-second** server-log timestamp. There is no per-client or per-target quota, throttle, or cost.
3. **No empty-pool fallback → hard denial.** When the pool is empty the server returns `err: "no one-time prekeys available for bob"` and logs `DENIED (no OPKs left)`. It does **not** fall back to a bundle without an OPK.

4.3 Impact and probes

Consequence — availability (DoS), not a downgrade. After draining Bob, real Alice tried to start a session with the offline Bob:

```
> Hello
09:37:56 fetch_bundle(bob): no one-time prekeys available for bob
```

The message is **never sent** — the client requires an OPK to run X3DH and has no signed-prekey-only fallback, so Alice is fully blocked. This is the security consequence: an anonymous attacker can deny *all* new sessions to an offline user for the price of a handful of unauthenticated requests, until Bob next republishes. Because the lab client refuses rather than silently proceeding without the OPK, this is a clean denial-of-service (and *not* a forward-secrecy downgrade — the implementation does not expose that weaker path).

Concurrency (spec A.3) — allocation appears atomic. `./attack1 -concurrent K` opens `K` sockets and releases all `K` fetches at the same instant (spin-flag barrier). Across repeated runs at $K = 2 \ldots 5$ the fetch *completion* order interleaves genuinely (e.g. request #4 returns before #1), confirming the requests do race — yet the handed-out ids are always a clean, duplicate-free `1..5`. No `opk_id` was ever served to two requests, so the server serializes allocation (single-threaded accept loop or a lock on the pool). *Caveat:* with only 5 OPKs over loopback the race window is small, so this is "no duplicate observed," not a proof that a non-atomic allocation is impossible on a busier server.

4.4 Proposed fix

```
# (1) Authenticate fetch_bundle. Require the requester to prove an identity
#     (the session the server already tracks per connection) before handing out
#     an OPK, and record which identity consumed which opk_id.
#
# (2) Rate-limit per requester AND per target. Cap OPKs handed out to any one
#     requester, and per target per time window, so one peer cannot drain a
#     user's whole pool; apply a small proof-of-work / backoff on bursts.
#
# (3) Do not consume the OPK at fetch time. Hand out the bundle but mark the OPK
#     "reserved"; only mark it "consumed" when a message that actually uses it
#     arrives (or expire the reservation on a timeout). A fetch that never leads
#     to a real session then costs nothing permanent.
#
# (4) Graceful degradation (defence in depth). If the pool is empty, fall back
#     to a signed-prekey-only X3DH bundle (SPK + IK, no OPK) rather than refusing
#     -- so exhaustion weakens one session's break-in secrecy instead of denying
#     service entirely. (Must be paired with (1)-(3); it is a fallback, not a
#     substitute for stopping the drain.)
#
# (5) Replenish / alert. Let a user top up OPKs out of band and warn when the
#     pool runs low, so an attack is visible and self-healing.
```

Fixes (1) and (2) stop the drain at its source; (3) removes the message-free consumption primitive; (4) ensures that even a fully drained user degrades rather than becomes unreachable.

5 Attack 2 — Malicious server / MITM (key substitution)

Scenario. Mallory *is* the server. She speaks the full wire protocol so Alice and Bob behave exactly as with the genuine server, but she substitutes key material during X3DH to break end-to-end encryption. Implementation: `attack2/attack2.cpp` (C++/OpenSSL), built in verified stages.

5.1 Trust model (A.1–A.4) — analysed from captures + client logs

Question	Finding	Evidence
A.1 How Alice gets Bob's IK	Only the server's `fetch_bundle` → `bundle`	it is the sole identity-key source on the wire (Phase 0 capture)
A.2 How IKs are authenticated	**They are not** — the SPK signature is not even checked	initiator logs `warn: SPK sig NOT verified`
A.3 Can the server swap key material undetected?	**Yes**	the bundle is unauthenticated relayed data; Alice accepts substituted keys
A.4 Independent verification mechanism	**None**	the client menu offers no fingerprint / safety-number comparison

The root cause is structural: X3DH binds identity keys only into the session associated data, and nothing anchors the identity key to the *real* peer. The spec's own premise — E2EE holds *"provided Alice and Bob have authentic copies of each other's identity keys"* — is unmet, because the only copy they ever get comes from the untrusted server.

5.2 The attack

1. **Substitute on fetch.** When Alice fetches `bob`'s bundle, Mallory returns her own X25519 keys (`ik_dh`, `spk`, `opk`) with a bogus `spk_sig` (not verified). Alice runs X3DH against Mallory and believes she is talking to Bob.

2. **Recover.** Alice's first frame carries the `init` block; Mallory runs `x3dh_respond` with her private keys, derives the same session key, and **decrypts Alice's plaintext**.

3. **Forward transparently.** Mallory runs `x3dh_initiate` against Bob's *real* bundle (which she holds as the server), re-encrypts the recovered plaintext, and delivers it to Bob, who decrypts it with his real keys and sees a normal message from "alice". Neither party detects anything.

4. **Two separate sessions, both directions.** Mallory keeps **two independent Double Ratchet sessions** alive for the whole conversation — one with Alice (Mallory as responder) and one with Bob (Mallory as initiator). Every frame is decrypted on the session it arrives from and re-encrypted on the other before forwarding, and each party's DH-ratchet turn is carried through on both sessions. Alice↔Bob therefore chat normally end-to-end while Mallory reads and relays **every message in both directions**. This is exactly the separation the task asks to demonstrate: *"establish separate sessions with Alice and Bob, decrypt messages received in each session, and forward messages to the other party."*

Because Ed25519 is never verified, the attack needs only four primitives: X25519, HKDF-SHA256, HMAC-SHA256, and ChaCha20-Poly1305-IETF.

5.3 Reverse-engineering the AEAD associated-data format (key milestone)

The one thing the spec leaves under-specified is `serialize(header)` and the exact byte layout of `st.AD`. We pinned it down empirically:

1. **Phase 0 capture** (passive relay `observe/relay.py` + `inspect.py`): recorded bundle sizes and the compact wire header `{"dh":...,"pn":...,"n":...,"init":...}`, with `init` only on message 0.

2. **First implementation** assumed the spec's literal `ad = st.AD + serialize(header)` with a compact header and raw `IK` ‖ `IK` prefix. An offline **self-test** (`-selftest`, simulating Alice and Bob with our own primitives) **passed**, proving the X3DH/ratchet/AEAD plumbing was correct.

3. **Live run falsified it:** `DECRYPT FAILED`. Self-test passing + live failing isolated the fault to **byte-format compatibility with the Go client**, not the crypto math.

4. **Discovery sweep:** since the first-message key is fixed once the session key is correct, we computed it once and brute-forced the associated data over a matrix of prefixes `{IK_dh raw/b64, IK_sig raw/b64, reversed, none}` × header serialisations `{compact, spaced, pn-n-dh, full-wire-incl-init, empty}`, reporting the first combination whose Poly1305 tag validated.

5. **Result:** `prefix = ik_dh raw`, `header = empty`. The associated data is exactly $\texttt{AD} = \texttt{ik_dh}_{\texttt{initiator}} \| \texttt{ik_d}$ (64 raw bytes); the header is **not** bound into the AEAD. This is a concrete divergence from

the spec pseudocode and also corrects the mechanism description in §3.2. A successful decrypt simultaneously confirmed that every X3DH/ratchet constant (`ikm = 0xFF*32 + ...`, `info = "AS3_X3DH"/"AS3_ROOT"`, zero salt/nonce, base64 standard-padded) matches the implementation.

6. **Applied to forwarding:** re-encrypting toward the real Bob with the same AD convention produced a frame Bob decrypted cleanly — confirming the end-to-end transparent MITM for the first message.

5.4 Result — full bidirectional MITM (verified on the live lab)

We built the attack in two verified stages and completed it:

- **Phase 2** — recover + forward Alice's *first* message only (single direction). Kept in the code as `-phase2` so the incremental story is demonstrable.
- **Phase 3 (final)** — persistent two-session relay reading the **whole conversation in both directions**. This is the default mode.

Live run (genuine Docker clients through the malicious server):

```
[attack2] stored REAL bob bundle (for forwarding)
[attack2] SUBSTITUTED bundle served for bob
[attack2] AD format discovered: prefix=dh_raw  header=empty
[attack2]  alice -> bob  #1 : "Hello Bob, What are you doing?"
[attack2]  bob -> alice  #1 : "I'm doing good, wbu"
[attack2]  alice -> bob  #2 : "I'm good as well. What are you upto these days?"
```

Both clients displayed every line normally; Alice saw only the `SPK sig NOT verified` warning she already ignores, and Bob saw ordinary messages from "alice". An offline multi-message self-test (`-selftest-full`) independently confirms the ratchet continuation, including each party's DH-ratchet turn (A→B, B→A, and A→B-after-ratchet all recovered and delivered intact).

Required evidence distinction (task B). The three cases are all observable:

Case	Example in this attack	How it shows up
Server **merely forwards**	any frame outside the MITM'd pair (e.g. a `poll`, or a message to a third user)	queued to the mailbox unchanged, no plaintext line printed
Server **recovers plaintext**	every Alice↔Bob frame, both directions	printed as `alice -> bob #n :  "..."` / `bob -> alice #n :  "..."` after `aead_decrypt` succeeds on the correct session
Fails authentication	a tampered/wrong-session frame, or — in `-phase2` mode — Bob's *reply*, which has no Mallory↔Alice sending session to match	`aead_decrypt` returns false → `AEAD authentication failed` on the client; Phase 3 removes this by maintaining both sessions

This directly answers task B: Mallory establishes **two separate sessions**, **decrypts in each**, and **forwards to the other party**, with the three evidence classes distinguished.

5.5 Proposed fix (Python-style pseudocode)

Root cause. Identity keys are delivered by the untrusted server and never authenticated out of band (§5.1). No server-side change helps, because the server *is* the adversary. The fix must let Alice and Bob confirm each other's identity key over a channel the server does not control, and bind the signed prekey to that authenticated identity.

```
# --- (1) Safety number: a short, comparable fingerprint of BOTH identity keys ---
# Mallory cannot forge it: she does not hold Alice's or Bob's real IK private key,
# and any key she substitutes changes the number both parties compute.
def safety_number(my_ik_pub, peer_ik_pub):
    a, b = sorted([my_ik_pub, peer_ik_pub])          # order-independent
    digest = SHA256(b"AS3-SAFETY-v1" + a + b)
    return base10_groups(digest[:30], groups=6)       # e.g. 6 x 5-digit blocks

# Alice and Bob read these aloud / scan a QR in person or over an already-trusted
# channel. If the numbers differ, a key was substituted -> abort.

# --- (2) Trust On First Use (TOFU) + change alerts, for when (1) is impractical ---
def on_bundle_received(peer, bundle):
    # (2a) Verify spk_sig against the IK *in the bundle* (necessary, not sufficient).
    if not ed25519_verify(bundle.ik_sig_pub, bundle.spk, bundle.spk_sig):
        abort("signed prekey signature invalid")

    pinned = trust_store.get(peer)                    # local, not server-controlled
    if pinned is None:
        trust_store.put(peer, bundle.ik_dh)           # pin on first contact
    elif pinned != bundle.ik_dh:
        # The identity key changed -> possible MITM or legitimate reinstall.
        alert_user("%s's identity key CHANGED -- verify safety number before sending"
                   % peer)
        require_manual_reverification(peer)           # block until user confirms

    # (2b) Bind the signed prekey to the PINNED identity, not just the bundle's.
    if not ed25519_verify(trust_store.get(peer), bundle.spk, bundle.spk_sig):
        abort("spk not signed by the pinned identity key")

# --- (3) Only after identity is trusted, run X3DH as before. ---
def start_session(peer, bundle):
    on_bundle_received(peer, bundle)                  # gate the whole handshake
    return x3dh_initiate(bundle)
```

Assumptions. (i) At least one out-of-band channel exists for the initial safety-number comparison (in person, phone, or a QR scan) — standard for Signal/WhatsApp. (ii) Each client has local, server-independent storage for pinned identity keys. (iii) Ed25519 signatures are actually checked (the current client skips this — see §5.1 — so fixing that is itself a prerequisite).

How Alice and Bob detect the malicious server. Under the substitution attack Alice is handed Mallory's `ik_dh`, so the safety number Alice computes ($\texttt{SHA256}(\texttt{IK}_{\texttt{alice}} \parallel \texttt{IK}_{\texttt{mallory}})$) differs from the one Bob computes ($\texttt{SHA256}(\texttt{IK}_{\texttt{mallory}'} \parallel \texttt{IK}_{\texttt{bob}})$); the mismatch on comparison reveals the MITM. With TOFU, the first session pins the real key and any later substitution raises the "identity key changed" alert. Either way the detection happens **off the server's path**, which is the only place a fix can live when the server is the attacker.

A Generative AI usage

Generative AI (Claude, via Claude Code) was used as an assistant throughout all three attacks: to analyse `protocol.md`, develop the proofs-of-concept in stages (Attack 0: transparent relay → single-field tampering → accumulate/flood amplifiers; Attack 1: single fetch → drain loop → concurrency/consequence probes; Attack 2: passive observation relay → honest server → key substitution + first-message recovery → full bidirectional two-session MITM), design the boundary and measurement experiments, and interpret the results.

Verification was empirical and adversarial to the model's own claims:

- Every prediction was tested against the running lab rather than accepted. For example, the model predicted `MAX_SKIP = 1000` would cause `-gap 1001` to be rejected with "too many skipped messages"; the experiment **disproved** this (the frame did full work and failed AEAD), and the report reflects the measured reality, not the prediction.
- All measured numbers (CPU %, memory, per-key cost) come from `docker stats` captures in `attack0/measure/`, not from the model.
- For Attack 1, the model predicted a possible non-atomic allocation race; the `-concurrent` probe **did not** reproduce a duplicate `opk_id`, and the report states the measured negative result (allocation appears atomic) with its caveat rather than the hypothesis. The DoS-vs-downgrade question was likewise settled by running real Alice against a drained Bob, not by assumption.
- The proof-of-concept code was compiled (`-Wall -Wextra`, no warnings) and run against the lab; observed client/server logs were cross-checked against the protocol pseudocode.
- For Attack 2, the model's *assumed* AEAD associated-data layout (literal `st.AD + serialize(header)`) **passed the offline self-test but failed the live decrypt**; the real format ($\texttt{AD} = \texttt{ik_dh}_{\texttt{initiator}} \parallel \texttt{ik_dh}_{\texttt{resp}}$ header not bound) was then recovered by a brute-force discovery sweep over the running lab's own ciphertext — a measured result, not a prediction. Both the Phase 2 single-message and Phase 3 bidirectional MITM were confirmed end-to-end on the live Docker clients, and an offline `-selftest-full` independently checks the ratchet continuation.

Representative prompts: "learn the assignment from the spec and protocol"; "implement the counter tampering as Step 2"; "what is the other implementation flaw that permits resource exhaustion?"; "implement accumulating and flood modes"; "analyse these `docker stats` results"; "make a skeleton for attack1 and proceed in steps"; "build the drain loop"; "add a concurrency probe"; "propose a plan to observe what Attack 2 needs"; "write the MITM that decrypts Alice's first message with neither party detecting it"; "complete the full bidirectional Mallory".